

תיעוד שיחת למידה - שלב 3

כתיבת, השלמת והרצת בדיקות יחידה ב-Java באמצעות JUnit

עבודה על הקבצים App.java ו-AppTest.java

נושא	בדיקות יחידה ב-Java בעזרת JUnit
מטרת השלב	לבדוק את הבדיקות הקיימות, להוסיף מקרי קצה חסרים ולהריץ את כל הבדיקות
תוצאה סופית	כל הבדיקות עברו בהצלחה והתקבלה ההודעה BUILD SUCCESSFUL

מטרת התיעוד

מסמך זה מתעד את תהליך הלמידה והעבודה בשלב 3 של התרגיל. בשלב זה נבדק הקובץ AppTest.java, הושווה לקוד שב-App.java, אותרו מקרים חסרים, נוספו בדיקות חדשות, תוקנה בדיקה בעייתית, ולבסוף כל הבדיקות הורצו באמצעות Gradle.

שלב א - הבנת המשימה

הסטודנטית: שאלתי מה בדיוק נדרש בשלב 3, מאחר שבקובץ AppTest.java כבר היו בדיקות קיימות.

העוזר: הוסבר שהמשימה היא לעבור על הבדיקות הקיימות, לוודא שכל הפונקציות מכוסות, לבדוק מקרי קצה וחריגות, להשלים בדיקות חסרות, להריץ את הבדיקות ולתעד את התהליך.

שלב ב - סקירת הקבצים

הסטודנטית: הצגתי את הקובץ App.java, שבו נמצאות 11 פונקציות: add, isPrime, reverse, factorial, isPalindrome, fibonacciUpTo, charFrequency, isAnagram, average, filterEvens ו-mostCommonWord.

העוזר: לאחר מכן נבדק AppTest.java ונמצא שלכל הפונקציות כבר קיימות בדיקות בסיסיות, אך היו כמה מקרי קצה שכדאי להוסיף ובדיקה אחת שדרשה תיקון.

שלב ג - תיקון הבדיקה mostCommonWord

העוזר: בבדיקה המקורית גם המילה hello וגם המילה world הופיעו פעמיים. לכן היה תיקון, ולא היה אפשר להבטיח שהפונקציה תחזיר דווקא world. הוחלט לשנות את הטקסט כך שהמילה hello תופיע שלוש פעמים.

```
@Test
public void testMostCommonWord() {
    String text = "Hello world! Hello everyone. Hello is common.";
    assertEquals("hello", App.mostCommonWord(text));

    String singleWord = "test";
    assertEquals("test", App.mostCommonWord(singleWord));
}
```

שלב ד - הוספת בדיקות חסרות

1. רשימה ריקה ב-filterEvens

הסטודנטית: הוספתי בדיקה שבודקת מה קורה כאשר מעבירים רשימה ריקה.

```
@Test
public void testFilterEvensWithEmptyList() {
    assertTrue(App.filterEvens(List.of()).isEmpty());
}
```

2. מספרים שליליים ואפס ב-filterEvens

העוזר: הוסבר שגם מספרים שליליים יכולים להיות זוגיים, וגם 0 הוא מספר זוגי. לכן נוספה בדיקה שמצפה לקבל את -2, -4 ו-0.

```
@Test
public void testFilterEvensWithNegativeNumbersAndZero() {
    List<Integer> numbers = Arrays.asList(-4, -3, -2, -1, 0);
    assertEquals(Arrays.asList(-4, -2, 0), App.filterEvens(numbers));
}
```

3. מערך עם איבר אחד ב-average

הסטודנטית: הוספתי בדיקה למערך שמכיל מספר אחד בלבד. במקרה כזה הממוצע צריך להיות אותו מספר.

```
@Test
public void testAverageWithSingleElement() {
    assertEquals(10.0, App.average(new int[]{10}), 0.001);
}
```

4. מחרוזות באורכים שונים ב-isAnagram

העוזר: נוספה בדיקה שמוודאת שמחרוזות באורכים שונים אינן יכולות להיות אנגרמות.

```
@Test
public void testIsAnagramWithDifferentLengths() {
    assertFalse(App.isAnagram("a", "aa"));
}
```

5. גבול 1 ב-fibonacciUpTo

הסטודנטית: הוספתי בדיקה לערך הגבולי 1. לפי מימוש הפונקציה, הרשימה המוחזרת היא 0, 1, 1.

```
@Test
public void testFibonacciUpToOne() {
    assertEquals(Arrays.asList(0, 1, 1), App.fibonacciUpTo(1));
}
```

שלב ה - בדיקת מבנה הקובץ

הסטודנטית: במהלך העריכה בדקתי שהבדיקות נמצאות בתוך המחלקה AppTest ושאינן סוגריים מסולסלים מיותרים.

העוזר: הוסבר שכל בדיקה חייבת להיות מסומנת ב-@Test, להיכתב בתוך AppTest, אך לא בתוך פונקציית בדיקה אחרת. כמו כן, הודגש שסידור ההזחה והשורות הריקות משפר את קריאות הקוד.

שלב ו - הרצת הבדיקות

הסטודנטית: לאחר שמירת הקובץ הרצתי את הבדיקות דרך הטרמינל.

```
./gradlew test
```

העוזר: ההרצה הסתיימה בהצלחה והתקבלה ההודעה BUILD SUCCESSFUL. משמעות הדבר היא שהקוד התקמפל וכל בדיקות היחידה עברו ללא כישלון.

שלב ז - סיכום הבדיקות שנוספו או תוקנו

פונקציה	מה נבדק	סוג הבדיקה
mostCommonWord	הסרת מצב של תיקו בין מילים	תיקון בדיקה קיימת
filterEvens	רשימה ריקה	מקרה קצה
filterEvens	מספרים שליליים ואפס	מקרה קצה

average	מערך עם איבר אחד	מקרה קצה
isAnagram	מחרוזות באורכים שונים	מקרה שלילי
fibonacciUpTo	גבול 1	מקרה קצה

מסקנה

בשלב 3 נלמד כיצד לא להסתפק בכך שקיימות בדיקות, אלא לבדוק את איכותן: האם הן חד-משמעיות, האם הן מכסות מקרים רגילים ומקרי קצה, והאם הן בודקות גם קלט לא תקין. לאחר התיקונים וההוספות כל הבדיקות עברו בהצלחה.